



UNIVERSITÀ DEGLI STUDI
DI PERUGIA

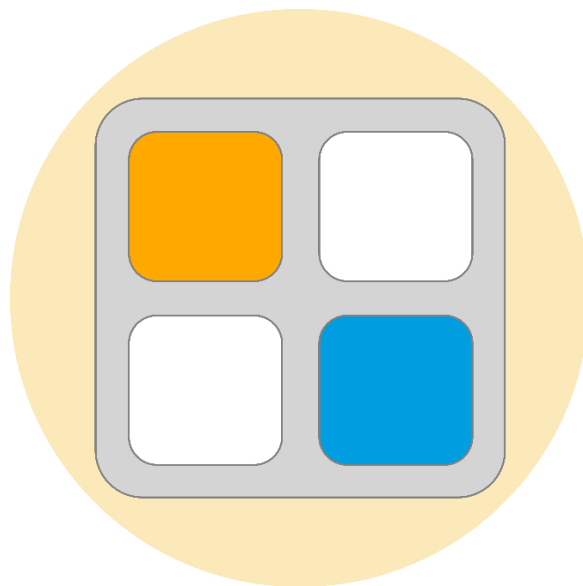
Tesina Finale di
Programmazione di Interfacce Grafiche e Dispositivi Mobili

Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2021-2022

DIPARTIMENTO DI INGEGNERIA

docente
Prof. Luca GRILLI

2048Colors
applicazione nativa per Android



Studente

Federico Ombelli

Sommario

1. DESCRIZIONE DEL PROBLEMA	3
DESCRIZIONE DEL GIOCO ORIGINALE	3
L'APPLICAZIONE 2048COLORS.....	3
2. SPECIFICA DEI REQUISITI.....	4
REQUISITI GENERALI.....	4
REQUISITI OPZIONALI.....	5
3. PROGETTO	6
ARCHITETTURA SOFTWARE.....	6
LOGIC (COLORS2048.LOGIC).....	7
VIEW (COLORS2048.VIEW)	8
PROBLEMI RISCONTRATI.....	12
4. POSSIBILITÀ DI ESTENSIONE E PERSONALIZZAZIONE.....	13
5. BIBLIOGRAFIA.....	13

1. Descrizione del problema

L'obiettivo di questo lavoro è la progettazione, lo sviluppo e l'implementazione di un'applicazione nativa per Android che proponga una versione rivisitata del noto videogioco per mobile 2048.

Descrizione del gioco originale

2048 è un videogioco single-player per piattaforma mobile diventato molto popolare dopo la sua pubblicazione nel 2014 inizialmente come gioco online poi ne sono state sviluppate numerose versioni ispirate al gioco originale.

È un gioco rompicapo il cui obiettivo è far scorrere delle tessere numerate su una griglia per unirle tra loro fino a formare una tessera con il numero 2048.

L'area di gioco è una griglia di formato 4x4 nella quale il giocatore fa scorrere le tessere numerate con una potenza del 2 tramite degli swipe sullo schermo in verticale o orizzontale, ad ogni swipe le tessere presenti sulla griglia si spostano nella direzione dello swipe nello spazio libero più vicino al bordo.

Ad esempio se un giocatore effettua uno swipe verso destra le tessere si spostano verso la casella libera più a destra dell'area di gioco, se è già presente una tessera sul bordo destro, l'altra si posizionerà subito a sinistra di essa.

Se due tessere contenenti lo stesso numero si scontrano, si fondono in un'unica tessera che avrà come numero la somma delle due tessere originarie.

Dopo ogni mossa una nuova tessera con il valore di 2 o 4 comparirà in una casella vuota casuale dell'area di gioco.

La partita finisce quando il giocatore riesce a creare la tessera con il numero 2048 o quando non ci sono più mosse disponibili.

L'applicazione 2048Colors

L'applicazione 2048Colors che si intende realizzare si ispira al gioco originale, riprendendone la meccanica di gioco ma è basato sui colori invece che sui numeri.

L'area di gioco è una griglia quadrata in cui si trovano tessere colorate con i diversi colori di una palette predefinita.

Il giocatore deve quindi muovere le tessere facendo degli swipe sullo schermo ma non può muovere una tessera singola, ad ogni swipe le tessere si muoveranno tutte insieme nella direzione dello swipe posizionandosi sulla casella libera più vicina al bordo della griglia.

Quando due tessere con lo stesso colore si scontrano si fondono a formare un'unica tessera con il colore successivo della palette.

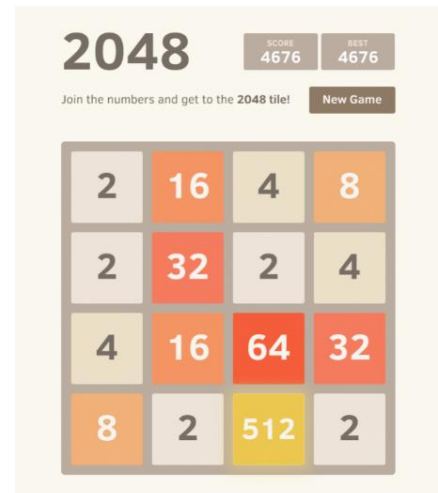


Figura 1 Schermata del gioco originale

Ad ogni mossa del giocatore viene generata una nuova tessera con il primo colore della palette in una posizione casuale della griglia ancora vuota.

Il giocatore vince se riesce ad ottenere la tessera con il colore finale della palette altrimenti il gioco termina quando non ci sono più spazi liberi sulla griglia.

2. Specifica dei requisiti

L'applicazione deve soddisfare i seguenti requisiti funzionali.

Requisiti Generali

- Presenza di un menu iniziale all'avvio dell'applicazione dal quale è possibile avviare il gioco e accedere alle altre sezioni dell'app: *Livelli*, *Opzioni*, *Tutorial*.
- La sezione *Livelli* deve dare al giocatore la possibilità di selezionare il livello di gioco per la partita, il livello scelto viene memorizzato per le partite seguenti anche se l'applicazione viene chiusa.
- La sezione *Opzioni* deve offrire al giocatore la possibilità di modificare la presenza della musica di sottofondo, degli effetti sonori e la dimensione della griglia di gioco.
- Quando il gioco viene avviato per la prima volta deve essere visualizzato un tutorial che spieghi l'obiettivo e le modalità di gioco, il tutorial non sarà più visualizzato nelle successive partite ma l'utente può visualizzarlo di nuovo accedendo alla sezione *Tutorial* nel menu principale.
- Il gioco deve prevedere 6 livelli in ordine di difficoltà, all'aumentare del livello di difficoltà aumenta la dimensione della palette colori e quindi il numero di colori possibili delle tessere. Il livello 1 ha 6 colori mentre il livello 6 ne ha 11, lo stesso numero del gioco originale.
- Presenza dell'audio di sottofondo durante il gioco.
- Presenza di effetti sonori durante le azioni di gioco principali (movimento delle tessere, unione di due tessere, livello superato, game over).
- L'applicazione deve prevedere uno stile grafico colorato e accattivante.
- Il giocatore controlla il movimento delle tessere tramite swipe sull'area di gioco che possono essere effettuati nelle 4 direzioni: verso l'alto, verso il basso, verso destra o verso sinistra.
- Il giocatore non può muovere una singola tessera ma ad ogni swipe muoverà tutte le tessere dell'area di gioco nella direzione dello swipe.
- L'interfaccia di gioco è suddivisa in 2 sezioni:
 - *Area di gioco*, posta nella sezione più bassa;

- *Area informativa*, posta nella sezione più in alto.
- L'*area informativa* deve fornire al giocatore informazioni sullo stato corrente della partita, in particolare deve indicare il punteggio, la palette colori completa del livello e il prossimo colore da raggiungere.
- Nella parte in alto a destra dell'*area informativa* è presente inoltre un pulsante per uscire dalla partita e tornare al menu principale.
- In caso di *game over* si apre una finestra popup che deve informare il giocatore della sconfitta e deve proporgli di riprovare il livello o di tornare al menu principale.
- In caso di *vittoria* si apre una finestra popup che deve informare il giocatore della vincita e deve proporgli di giocare al livello successivo o di tornare al menu principale.

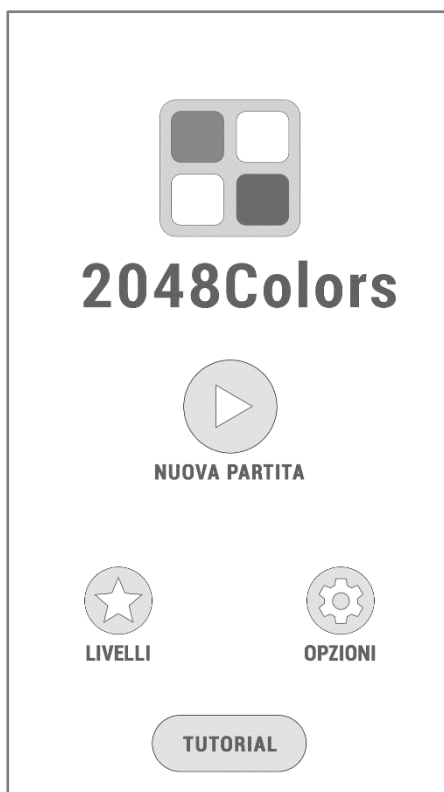


Figura 2 Wireframe menù iniziale

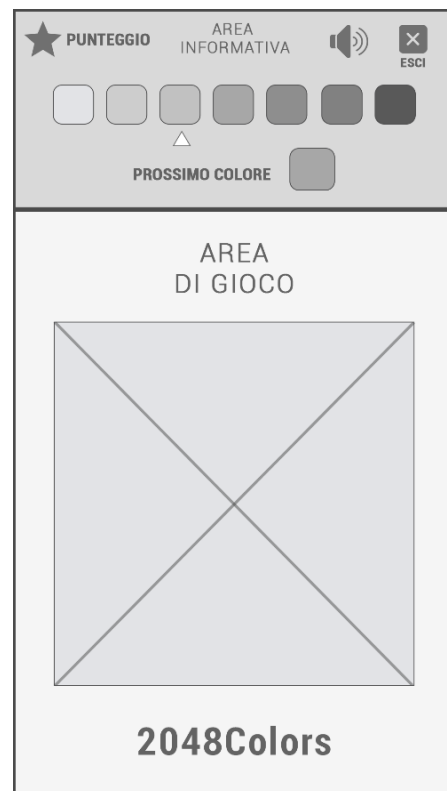


Figura 3 Wireframe schermata di gioco

Requisiti opzionali

- Possibilità di salvare lo stato della partita in corso in caso di uscita dal gioco.
- Possibilità di modificare la dimensione della griglia di gioco (*implementato*).
- Possibilità di salvare i punteggi migliori per ogni livello di gioco.

3. Progetto

Architettura software

Per la realizzazione dell'applicazione *2048Colors* è stato utilizzato il pattern architetturale **Logic-View** che prevede un disaccoppiamento delle classi logiche da quelle grafiche.

Il **Logic** si occupa infatti di gestire soltanto la logica di base del videogioco e mantenere lo stato delle variabili in un dato istante. Il **View** si occupa di gestire il rendering dell'applicazione sullo schermo e registrare gli input dell'utente.

Logic e View comunicano tra loro grazie a 2 interfacce: *IView* e *ILogic*.

- L'interfaccia **IView** espone i metodi che le classi del Logic possono invocare per richiedere aggiornamenti della visualizzazione alle classi grafiche in risposta al verificarsi di determinate condizioni (ad esempio il Logic comunica che la configurazione della griglia di gioco è cambiata al View che provvede ad aggiornare la visualizzazione).
- L'interfaccia **ILogic** in modo analogo raccoglie i metodi che le classi del View possono invocare per ricevere aggiornamenti dal Logic e per comunicare gli input dell'utente.

Il diagramma UML nella figura seguente rappresenta l'architettura software ad alto livello, con le classi e i package principali e le dipendenze tra di essi, si evidenziano inoltre le due interfacce *IView* e *ILogic*.

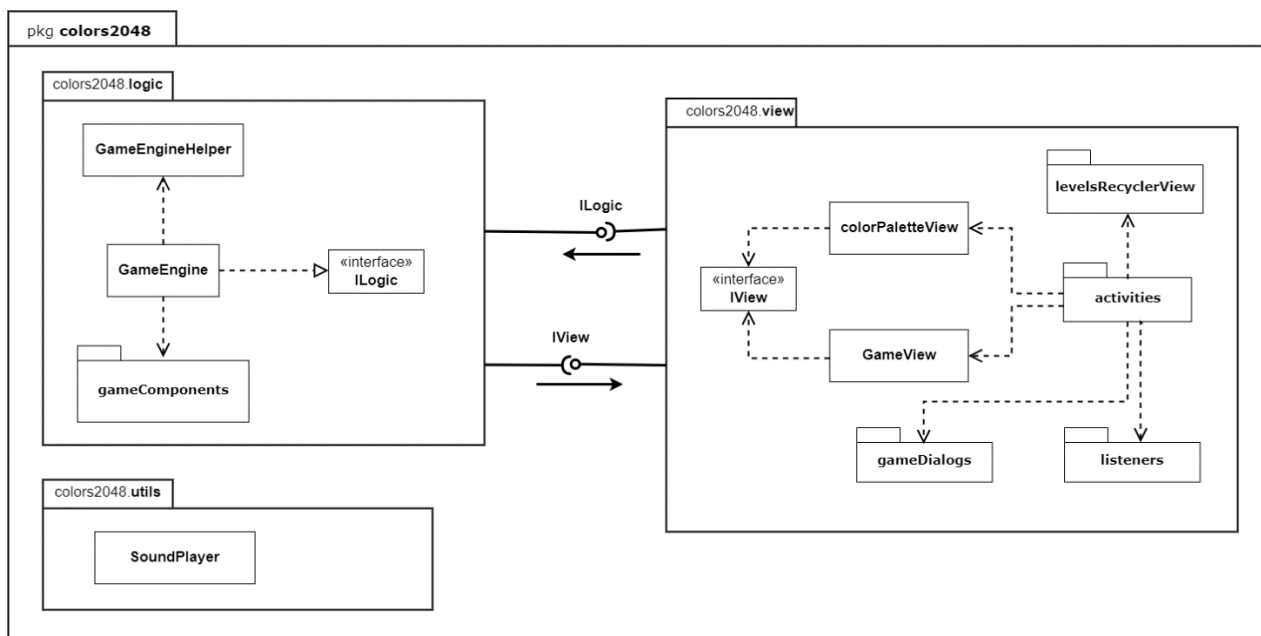


Figura 4 Diagramma UML che descrive la struttura dei package e la comunicazione tra essi

L'applicazione contiene inoltre un package *Utils* che racchiude le classi per la gestione dell'audio e degli effetti sonori.

Di seguito viene descritta nel dettaglio la struttura dei due package *View* e *Logic*.

Logic (*colors2048.logic*)

Le classi del *Logic* sono racchiuse nel package *colors2048.logic* e sono rappresentate nel diagramma UML seguente.

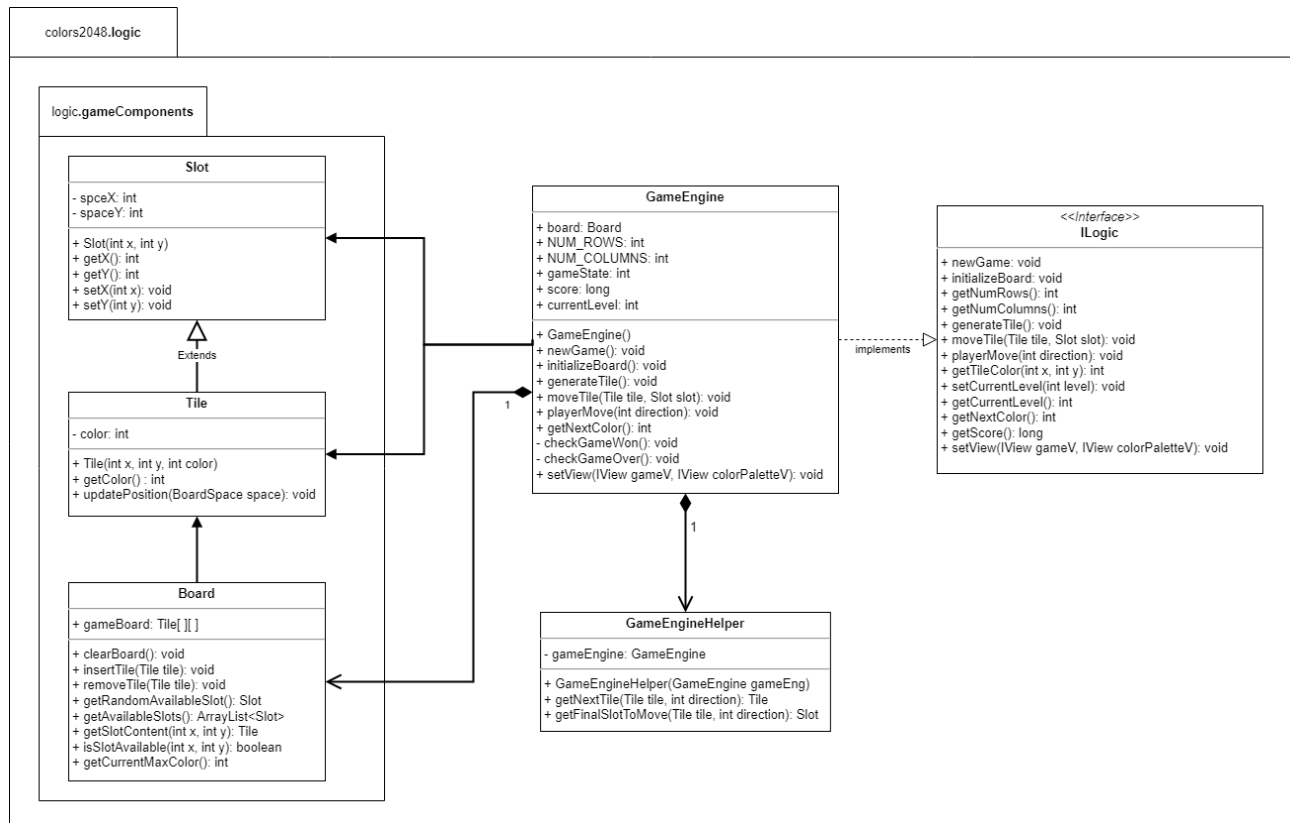


Figura 5 Diagramma UML di classe del package logic

Nel package sono presenti la classe principale *GameEngine*, la classe di supporto *GameEngineHelper* e un package interno *logic.gameComponents*.

Le classi del package **logic.gameComponents** implementano i componenti logici fondamentali del gioco:

- **Slot** rappresenta un singolo spazio della griglia di gioco memorizzandone le sue coordinate;
- **Tile** rappresenta una singola tessera posizionata sulla griglia di gioco, estende *Slot* e memorizza il colore della tessera con un numero intero progressivo a partire da zero;
- **Board** rappresenta la griglia di gioco e mantiene la posizione di tutte le tessere presenti sulla griglia tramite un array bidimensionale di oggetti *Tile*, contiene in particolare il metodo *getRandomAvailableSlot()* che restituisce uno spazio libero casuale della griglia ed è invocato dal *GameEngine* dopo ogni mossa per generare una nuova tessera.

La classe principale **GameEngine** contiene la maggior parte della logica di funzionamento del gioco e memorizza le variabili quali il punteggio, il livello, lo stato del gioco (attivo / interrotto / vittoria / sconfitta) ad un dato istante.

Implementa inoltre l'interfaccia **ILogic** che permette la comunicazione con le classi del View. Questa contiene in particolare il metodo `setView()` che viene invocato dalla *GameActivity* per passare al Logic il riferimento agli oggetti delle due classi view principali *GameView* e *ColorPaletteView*.

I metodi più importanti della classe *GameEngine* sono: `newGame()` per avviare una nuova partita, `checkGameWon()` e `checkGameOver()` che verificano rispettivamente se la partita in corso è stata vinta o persa e in caso positivo lo comunicano al View e `playerMove(int direction)` che merita un'analisi più approfondita.

Il metodo `playerMove` infatti è il metodo più esteso della classe ed è invocato dalle classi del View quando il giocatore effettua una mossa in una determinata direzione. In funzione della direzione ricevuta (*up, down, right, left*) determina la posizione finale di ogni tessera presente sulla griglia di gioco e verifica l'eventuale unione di due tessere uguali adiacenti.

Fa uso inoltre di una classe di supporto **GameEngineHelper** che fornisce due ulteriori metodi importanti: `getNextTile(Tile tile, int direction)` che restituisce la prossima tessera nella direzione della mossa rispetto a quella passata in input e `getFinalSlotToMove(Tile tile, int direction)` che calcola l'ultimo slot libero della griglia nella direzione data nel quale poi sarà mossa la tessera.

View (*colors2048.view*)

Le classi del View sono racchiuse nel package *colors2048.view* e suddivise in ulteriori package interni.

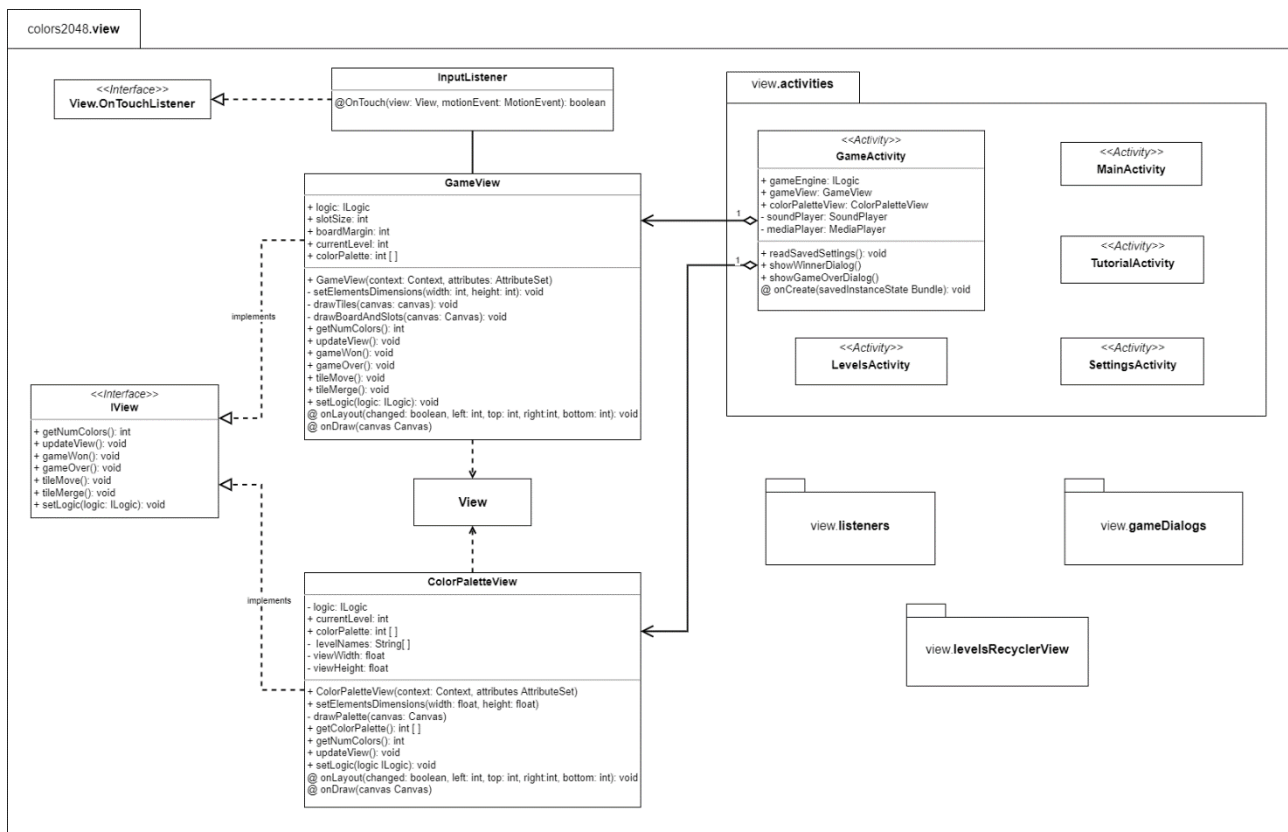


Figura 6 Diagramma UML del package view che evidenzia le classi responsabili del rendering del gioco.

Il diagramma UML di classe in *figura 6* mette in evidenza le classi che si occupano del rendering della schermata di gioco, parte fondamentale dell'applicazione, quelle contenute negli altri package saranno analizzate in seguito.

GameView e *ColorPaletteView* estendono *View*, implementano l'interfaccia *IView* e rappresentano le due sezioni principali in cui è divisa la **GameActivity**. Le due view sono istanziate dalla *GameActivity* sulla base del layout descritto tramite file xml e i loro riferimenti sono passati al Logic utilizzando il metodo *setView()* dell'interfaccia *ILogic*.

- La **GameView** si occupa del rendering dell'*area di gioco*, calcola le dimensioni della griglia e delle tessere dinamicamente facendo l'override del metodo *onLayout(...)* e le disegna facendo l'override del metodo *onDraw(...)*, utilizzando i metodi messi a disposizione dalla Canvas data la semplicità delle forme da rappresentare.

Per rappresentare correttamente i colori la *GameView* legge per ciascuna tessera il suo codice colore come codificato nel Logic e recupera poi il relativo codice in formato esadecimale tramite il metodo *getColorPalette()* della *ColorPaletteView*.

- La **ColorPaletteView** si occupa infatti del rendering dell'*area informativa*, rappresentando la palette colori del livello, il prossimo colore da raggiungere e le informazioni testuali quali il punteggio e il nome del livello.

Per rappresentare la palette colori recupera il livello corrente dal Logic e legge il file xml *levelsColorPalette.xml* che contiene le informazioni sul numero dei colori per ciascun livello e i codici colore esadecimali della palette globale, in questo modo calcola dinamicamente le dimensioni e il colore di ciascuna tessera e le disegna facendo l'override del metodo *onDraw(...)* in modo analogo alla *GameView*.

Una schermata della *GameActivity* nella sua versione finale con il gioco in azione è visualizzata in *figura 7*.



Figura 7 Schermata di gioco

L'input dell'utente viene registrato dalla *GameView* che si serve come listener della classe **InputListener**. *InputListener* estende *OnTouchListener* e permette di riconoscere gli swipe dell'utente sullo schermo, in risposta ad essi invoca il metodo *playerMove()* del Logic passandogli la direzione dello swipe.

Nel package *view* sono presenti anche altri quattro package interni:

- *view.activities* contiene le classi delle Activity di Android;
- *view.gameDialogs* contiene le classi che implementano i popup di vittoria e sconfitta visualizzati al termine di una partita;
- *view.listeners* contiene le interfacce utilizzate per inviare e ricevere gli eventi di gioco tra le classi del View;

- *view.levelsRecyclerView* contiene le classi che implementano la schermata dinamica di selezione del livello presente nella *LevelsActivity*.

Il package **view.activities** contiene le 5 classi delle Activity di Android utilizzate all'interno dell'applicazione, tutte estendono l'activity di base AppCompatActivity e utilizzano layout basati su file descrittori xml.

Il diagramma UML seguente mostra nel dettaglio il package *view.activities* e *view.levelsRecyclerView*.

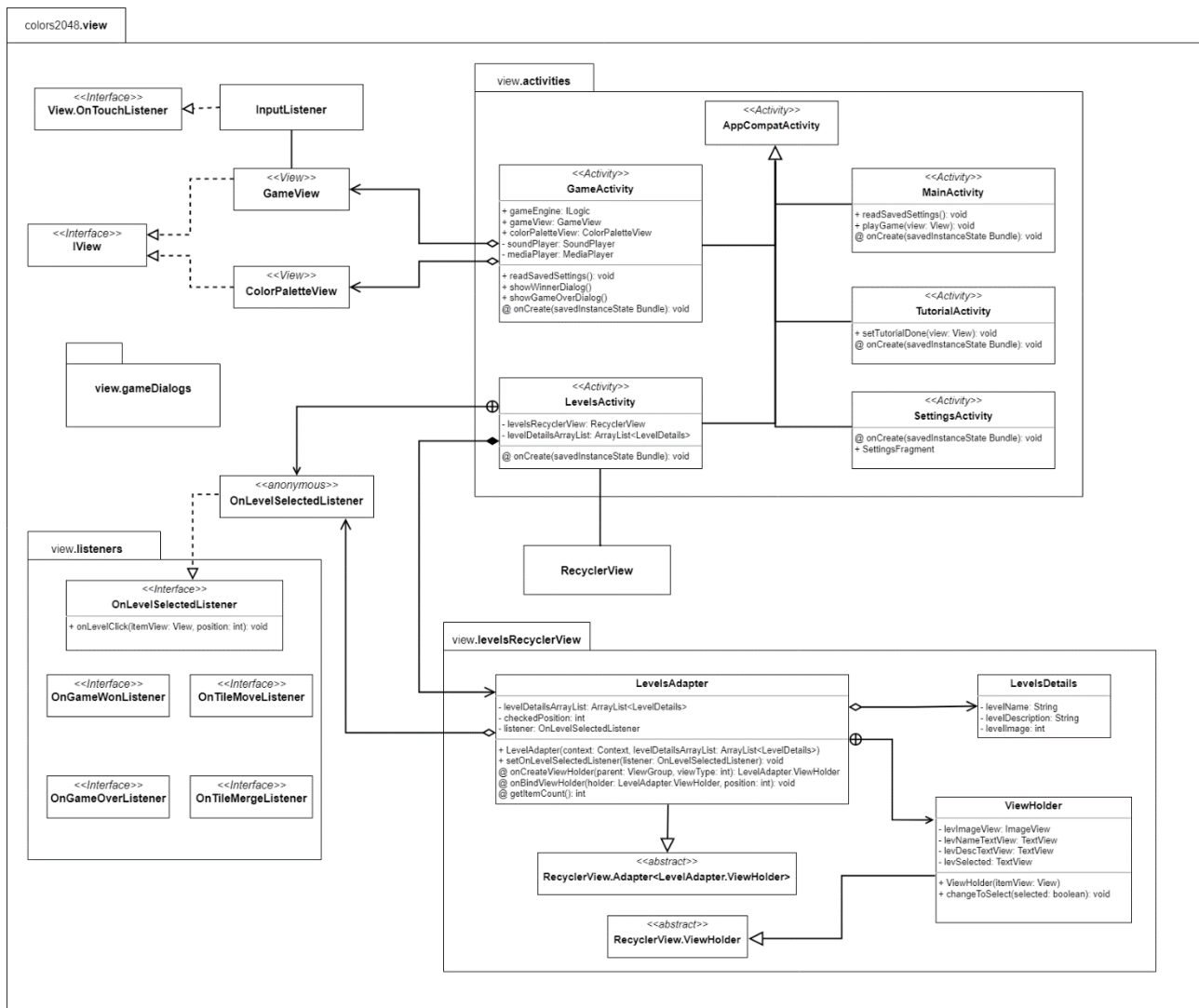


Figura 8 Diagramma UML che evidenzia la struttura dei package activities e levelsRecyclerView

La **MainActivity** è l'activity "launcher" e si apre all'avvio dell'applicazione da parte dell'utente, permette l'avvio delle altre activity alla pressione degli appositi pulsanti utilizzando degli Intent.

La **GameActivity** rappresenta la schermata di gioco principale ed è già stata descritta in precedenza.

La **SettingsActivity** contiene le impostazioni di gioco e si apre in modalità pop-up. Le impostazioni sono realizzate tramite la classe *SharedPreferences* della libreria *Android Jetpack* e possono essere lette e scritte da tutte le activity.

La **TutorialActivity** è una semplice activity che contiene soltanto il tutorial del gioco, viene visualizzata solo al primo avvio del gioco da parte dell'utente, grazie a una impostazione specifica oppure può essere richiamata dal pulsante sulla MainActivity.

La **LevelsActivity** ha richiesto un'implementazione più complessa poiché per la selezione del livello si è scelto di non utilizzare un semplice selettore ma una lista dettagliata di ogni livello che specifichi il numero dei colori e visualizzi anche un'anteprima della palette.

Di seguito ne viene descritta nel dettaglio l'implementazione.

Si è deciso di utilizzare a questo scopo una *RecyclerView* di Android, un widget della libreria *Jetpack* che permette di visualizzare una lista dinamica di elementi.

I dettagli di ogni livello sono memorizzati nel file *arrays.xml* come 3 array di stringhe, uno per i nomi dei livelli, uno per le descrizioni dei livelli e uno per i nomi delle immagini di preview delle palette colori, memorizzate come file vettoriali nella cartella *drawable*.

Per l'implementazione è stata creata la classe **LevelDetails** i cui oggetti mantengono le informazioni di ciascun livello e la classe **LevelAdapter** che estende *RecyclerView.Adapter<LevelAdapter.ViewHolder>*.

Quest'ultima si occupa di:

- creare una view per ogni scheda livello a partire da un file di layout xml tramite l'override del metodo *onCreateViewHolder()*;
- leggere le informazioni da *arrays.xml* e popolare una lista con tanti oggetti *LevelDetails* quanti sono i livelli ciascuno con i relativi dati come attributi;
- personalizzare ogni view con i dati letti e associarla ad oggetti *LevelAdapter.ViewHolder*.

LevelAdapter.ViewHolder è una classe interna a *LevelAdapter* ed estende la corrispondente classe *RecyclerView.ViewHolder* ed è utilizzata per mantenere la view di ogni scheda e ricevere gli input dell'utente aggiornando la variabile con il livello selezionato e notificando alla *RecyclerView* di aggiornare la vista.

Per la comunicazione tra l'activity e gli oggetti *ViewHolder* viene utilizzata un'interfaccia apposita *OnLevelSelectedListener* con il metodo *onLevelClick()*, che viene impostata come attributo *onClickListener* della vista mantenuta dal *ViewHolder*.

In questo modo al click sulla vista la *LevelsActivity* riceve l'input e imposta il livello selezionato nelle *SharedPreferences*.

Questo approccio che fa uso dei *Listener* viene impiegato anche per gestire la visualizzazione dei Dialogs in risposta ad eventi di gioco. Le classi che gestiscono questi aspetti fanno parte dei package **view.listeners** e **view.gameDialogs** e la loro struttura è rappresentata nel diagramma UML in *figura 9*.

I Dialogs sono creati estendendo la classe *DialogFragment* della libreria *Jetpack* e sono istanziati dalla *GameActivity* facendo uso della classe *FragmentManager*. Quest'ultima permette anche di gestire gli input dell'utente sul Dialog e riportare l'evento in risposta all'activity che può quindi avviare apposite azioni in risposta.

Una descrizione più approfondita sull'utilizzo dei *Listener* si trova nel paragrafo seguente che tratta i problemi riscontrati nello svolgimento del progetto.

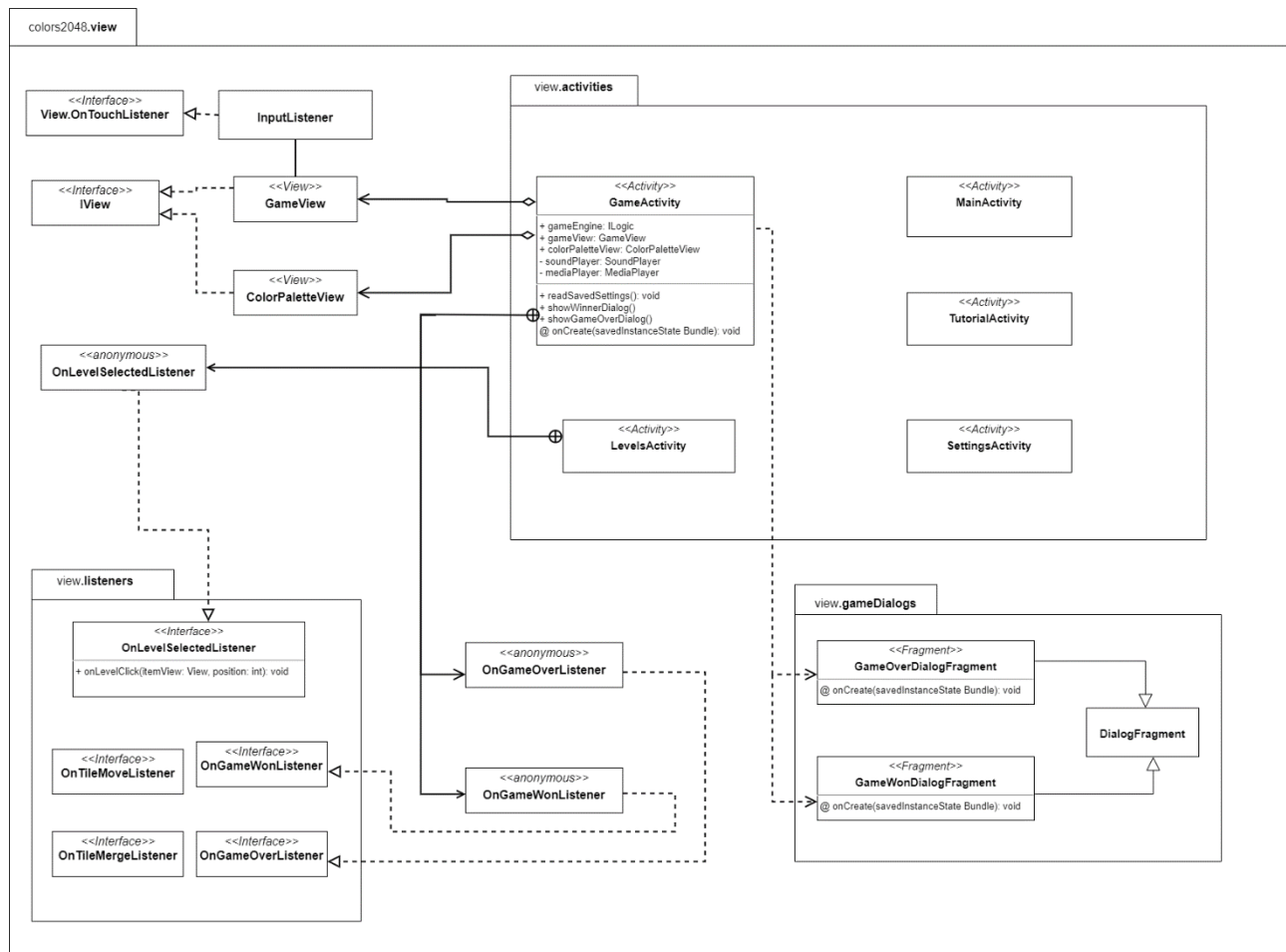


Figura 9 Diagramma UML che evidenzia la struttura dei package listeners e gameDialogs

Problemi Riscontrati

L'applicazione *2048Colors* non ha una logica di gioco particolarmente complessa e l'implementazione delle classi logiche non ha dato particolari problemi ad eccezione del calcolo della posizione finale delle tessere.

Questo ha richiesto infatti di gestire le operazioni in modo diverso a seconda della direzione dello *swipe* fatto dall'utente e ciò risultava in un metodo eccessivamente lungo. Il problema è stato risolto con un approccio modulare creando due metodi più brevi ciascuno dedicato a un'operazione specifica che vengono richiamati dal metodo principale. I metodi di supporto sono stati inseriti in una classe separata *GameEngineHelper* per ridurre ancora di più le dimensioni della classe principale.

Un altro aspetto che ha dato alcuni problemi è stato quello della comunicazione tra View e Activity degli eventi di gioco, ad esempio la vittoria e la sconfitta. Infatti in risposta a questi eventi la *GameActivity* deve visualizzare un *Dialog* con il messaggio di vittoria o sconfitta e le azioni a disposizione dell'utente, inoltre in seguito a una mossa riuscita o all'unione di due tessere uguali deve riprodurre degli effetti sonori.

Questa problematica è stata risolta utilizzando i *Listener*, interfacce che vengono implementate mediante oggetti anonimi direttamente all'interno della *GameActivity* e sono poi passati come riferimento alla *GameView*. Quindi al verificarsi di un evento, ad esempio la sconfitta, il *Logic* invoca il metodo *gameOver()* sulla *GameView* utilizzando l'interfaccia *IView*, la *GameView* quindi invoca il metodo *onGameOver()* sull'oggetto che implementa l'interfaccia *OnGameOverListener*. L'Activity in questo modo riceve l'evento e può visualizzare il Dialog corrispondente o riprodurre l'effetto sonoro.

4. Possibilità di estensione e personalizzazione

Il gioco è stato progettato con la possibilità di estenderlo o personalizzarlo in modo semplice e senza modifiche sostanziali del codice sorgente.

In particolare è stata prevista la possibilità di aggiungere nuovi livelli, di cambiare i colori e le dimensioni della palette colori di ciascun livello.

Per effettuare queste modifiche è sufficiente infatti modificare due file xml contenuti nella cartella *res/values*:

- il file *levelsColorPalette.xml* che contiene il numero dei livelli del gioco, un array di integer in cui l'indice dell'array rappresenta il livello e gli elementi danno il numero di colori per livello e un array che contiene i colori in formato esadecimale della palette globale del gioco.
- il file *arrays.xml* che contiene un array di stringhe con i nomi dei livelli, uno con le descrizioni e uno con i nomi delle immagini da utilizzare come preview per le palette colori di ciascun livello. Le nuove immagini devono essere aggiunte nella cartella *res/drawable* nel formato vettoriale utilizzato da Android.

5. Bibliografia

- wikipedia.org, "2048 (videogame)", [https://en.wikipedia.org/wiki/2048_\(video_game\)](https://en.wikipedia.org/wiki/2048_(video_game)).